UNIVERSITY OF GENOVA

Assignment D3-V2: Warehouse Robotics – Multi-Robot Coordination

Mantovani Luca

June 15, 2026

1 INTRODUCTION

The primary objective of this report is to investigate multi-agent coordination in a shared environment using automated planning techniques. The operational scenario focuses on a warehouse where multiple identical robots are tasked with transporting packages concurrently across a shared spatial graph. A critical challenge arises in this setting: robots must navigate and complete their deliveries without occupying the same physical location simultaneously. A core requirement of this analysis is the avoidance of implicit coordination; instead, the goal is to define rules and constraints from which cooperative behavior and spatial awareness can emerge naturally.

To explore this problem, the project requires the development and comparison of two distinct modeling approaches using the Planning Domain Definition Language (PDDL) and its temporal extension, PDDL+. The first phase involves designing a basic, sequential PDDL domain to understand how multi-agent constraints can be explicitly modeled in a discrete time-step environment. The second phase advances the scenario by transitioning to PDDL+, introducing continuous temporal dynamics and event-driven mechanics to represent the actual duration of actions and the physical realities of spatial blocking.

Finally, to rigorously evaluate and validate both approaches, specific problem instances were formulated for each domain. Solving these problem instances is essential to verify that the domains function correctly and to demonstrate how temporal and sequential aspects influence multi-agent coordination under varying degrees of complexity.

2 Q1 – BASIC PDDL MODEL

The first phase of the assignment required the development of a standard, sequential PDDL domain (`concurrent_warehouse`) to handle multi-agent coordination.

2.1 DOMAIN CHARACTERISTICS AND EXPLICIT MODELING

To satisfy the modeling guidelines, the domain relies on atomic actions and explicit multi-agent constraints. Robot, location, and package identities are explicitly modeled using the `robot`, `location` and `package` types, meaning every action is parameterized to a specific agent (`?r - robot`) and/or a specific location (`?l - location`) and/or a specific package (`?p - package`).
Occupancy and capacity constraints are enforced through two predicates:

- `occupied ?l`: Tracks whether a specific location in the spatial graph is currently occupied.
- `robot_free ?r`: Ensures a robot is not already carrying a package before it can initiate a `pick_up` action.

The remaining predicates are defined as follows:

- `carry ?r ?p`: Represents the assignment of a package to a robot, indicating that the robot is currently transporting that package.
- `in ?r ?l`: Denotes the current location of a robot within the warehouse graph.
- `in_pkg ?p ?l`: Represents the current location of a package.
- `connect ?l1 ?l2`: Defines bidirectional connectivity between two locations in the environment graph.

These predicates define the spatial and relational structure of the domain. Each robot is constrained to occupy exactly one location at a time, while packages are initially placed within locations. Movement is only possible between connected locations, as specified by the `connect` relation. When a robot is co-located with a package and satisfies the necessary preconditions, it can initiate a `pick_up` action and start carrying it. Each robot is restricted to carrying at most one package at a time.
The core of the collision avoidance system is embedded within the `move` action. By utilizing the `:negative-preconditions` requirement, a robot can only transition to a new location if it explicitly evaluates `(not (occupied ?l2))`. Upon execution, the action's effects atomically transfer the robot, freeing the previous location `(not (occupied ?l1))` and occupying the new one `(occupied ?l2)`. This discrete logic guarantees that no two agents can ever share a spatial node simultaneously.
The remaining two actions are `pick_up` and `drop`, which are used respectively to load a package onto a robot and unload it from the robot.

2.2 Problem Instances

To validate the domain and demonstrate how constraints dictate robot behavior, two distinct problem instances were constructed and evaluated.

2.2.1 Independent Task (`indipendent_task`)

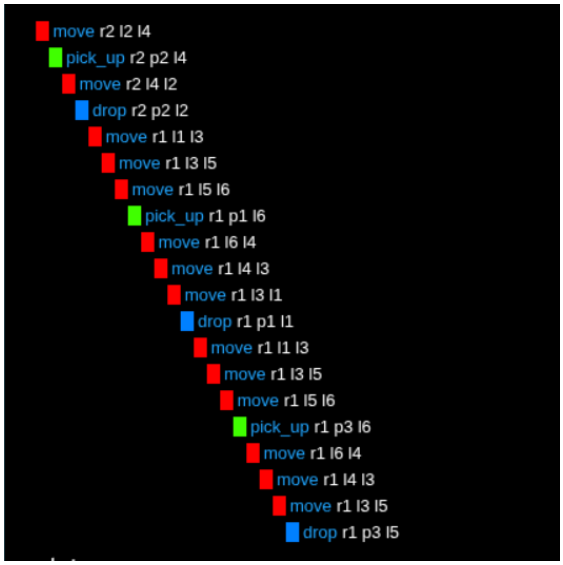
This problem tests the baseline functionality of the domain by placing two robots (`r1`, `r2`) in isolated sub-graphs.

- **Environment:** The graph is physically segregated. `r1` operates exclusively between `l1` and `l3` to retrieve `p1`, while `r2` operates between `l2` and `l4` to retrieve `p2`.
- **Coordination Analysis:** Because their paths and objectives do not intersect, no resource contention occurs. The planner generates a straightforward sequence of operations, interleaving the actions of `r1` and `r2` arbitrarily. The `(not (occupied ?l2))` precondition is never threatened by the other agent, proving that the domain efficiently handles parallel, non-interfering tasks without unnecessary delays.

2.2.2 Concurrent Interacting Task(`concurrent_task`)

This problem introduces a highly interconnected spatial graph where interference is inevitable. Two robots must retrieve three packages (`p1`, `p2`, `p3`) scattered across deeper nodes (`l4`, `l6`).

- **Environment:** The graph contains loops and shared critical pathways. Both robots start at adjacent nodes (`l1` and `l2`) and must traverse these shared spaces to reach their respective goals.
- **Coordination Analysis:** In this highly congested scenario, effective coordination emerges entirely from the domain's constraints. If `r1` plans a route through `l4`, but `r2` is currently occupying it, the planner cannot execute the `move` action for `r1`. Because standard PDDL evaluates actions sequentially, the planner acts as a traffic controller: it is forced to generate a serialized plan where `r1` either waits (its actions are delayed in the sequence) or takes an alternate route until `r2` moves and triggers the `(not (occupied l4))` effect.



3 Q2 – PDDL+ Model

While the basic PDDL model guarantees collision-free navigation, it treats actions as instantaneous state transitions. In reality, warehouse robots take time to traverse distances and load/unload packages. To model this correctly, the domain was upgraded to PDDL+, introducing continuous temporal dynamics, numeric fluents, and event-driven mechanics to represent the physical constraints of the warehouse accurately.

3.1 Iterative Design and Previous Formulations

Before arriving at the final PDDL+ model, the domain underwent several changes.

- **Domain 1 (Durative Actions):** The first prototype (`concurrent_warehouse2`) attempted to encapsulate movement into a single `durative-action`. It checked if a target location was empty at the `start` of the action and evaluated an `enter` event at the end. However, this created severe race conditions: multiple robots could simultaneously begin moving toward the same empty node, resulting in artificial deadlocks upon arrival.

- **Domain 2 (The Waiting Rooms):** To resolve arrival collisions, a second prototype introduced a new structural type: the `waiting-room`. Robots did not move directly between locations, but into an abstract buffer associated with the destination. While this prevented planner deadlocks, it fundamentally violated the core modeling guidelines. Introducing an infinite-capacity abstract buffer altered the shared physical graph, acting as an artificial loophole rather than genuinely solving spatial interference without implicit coordination. Furthermore, the domain isn't complete and functional because I realized the error before finishing it.

3.2 THE FINAL ARCHITECTURE

The finalized domain (`concurrent_warehouse_official`) completely removes abstract buffers and rigid durative actions. It just uses continuous numeric processes and discrete events.

3.3 SPATIAL ABSTRACTION

To fully understand the domain's behavior, it is crucial to clarify the spatial abstraction used in the model. In the context of this warehouse scenario, a `location` does not represent a physically enclosed "room" with doors, but rather a specific coordinate point, workstation, or intersection within the open warehouse floor.
Because occupancy constraints (`occupied`) are applied exclusively to these discrete nodes, the connections (`connect`) representing the spaces between them do not have capacity limits. This design choice reflects a realistic industrial environment where spaces are sufficiently wide, allowing two or more robots to travel between two different locations using the same space simultaneously without colliding.

3.4 EVENT-DRIVEN TEMPORAL MECHANICS

To correctly model continuous time and physical constraints, the PDDL+ domain replaces instantaneous transitions with a decoupled architecture of actions, continuous processes, and autonomous events.
New Predicates and Numeric Fluents: Robot states are dynamically tracked using new transitional predicates: `moving`, `blocked`, `busy`, `picking`, and `dropping`. The physical rules of the environment are dictated by numeric functions: `distance` defines the length of corridors, `speed` determines agent capabilities, `distance-remaining` tracks real-time progress, and `operation-timer` manages task durations.
Movement and the Blocking Mechanic: Navigation is handled by continuous spatial tracking rather than discrete jumps:

- **Initiation:** The `start-moving` action marks the robot as `moving` and clears its previous location. This activates the `move_process`, which continuously decreases the `distance-remaining` based on the robot's specific `speed`.
- **Arrival Events:** Once the robot completes its journey (`distance-remaining` ≤ 0), the environment automatically resolves the outcome via events. If the target location is free, the `enter` event triggers, placing the robot inside and marking the node as `occupied`.
- **Queueing (`block`):** If the target point is currently occupied, the `block` event inevitably triggers. The robot does not retreat or enter an artificial waiting area; instead, it halts exactly at the threshold of the node, entering a `blocked` state. It waits in the adjacent open space of the corridor until the area is cleared, at which point the planner can explicitly schedule the `unblock-and-enter` action to resolve the queue.

Continuous Package Handling: Package retrieval and delivery are modeled as time-consuming operations rather than instantaneous effects. The `start-pick-up` and `start-drop` actions restrict the robot by marking it as `busy` and activate the `picking_process` and `dropping_process`, which continuously increase the `operation-timer`. Once the timer reaches the required operational threshold (10 time units), the `end-pick-up` or `end-drop` events automatically trigger to finalize the state transition, load/unload the package, and free the robot for its next task.

3.5 PROBLEM INSTANCES

To test the temporal mechanics and the robustness of the blocking constraints, two bottleneck scenarios of increasing complexity were created and analyzed.

3.5.1 BASELINE BOTTLENECK: TWO-ROBOT CONTENTION

The first scenario (`test_warehouse_bottleneck`) introduces a basic contention. Two robots start in different rooms (`r1` in `l1`, `r2` in `l4`) but must converge on a single central hub (`l2`) to retrieve their respective packages (`p1`, `p2`) and deliver them to `l3`.
An analysis of the generated output plan reveals that the GBFS heuristic engine immediately deploys sophisticated emergent behaviors to handle the contention:

- **Heuristic Delay:** At $t = 0.0$, the slower robot `r1` begins moving toward `l2`. However, the faster robot `r2` is intentionally held back at `l4` until $t = 7.0$. The planner calculates that moving `r2` immediately would lock the intersection prematurely, stalling the heuristic descent.
- **Active Yielding:** At $t = 11.0$, `r1` arrives at the central hub. Instead of initiating a loading operation that would block the faster robot, the planner forces `r1` to actively yield the room by moving into an adjacent corridor (`start-moving r1 l2 l4`). This instantly clears the space, allowing `r2` to execute `(unblock-and-enter r2 l2)` and retrieve its package. Once `r2` leaves at $t = 21.0$, `r1` returns to the hub to complete its task.
- **Explicit Queueing:** The explicit physical queue is flawlessly demonstrated at the delivery node `l3`. At $t = 45.0$, `r2` begins a 10-tick drop operation inside `l3`. During this time, `r1` arrives. Because the room is occupied, `r1` is forced into a blocked state outside the node. Exactly at $t = 55.0$, when `r2` finishes and leaves, the planner schedules the explicit `(unblock-and-enter r1 l3)` action, simulating a perfect waiting line.

```
Found Plan:
0: (start-moving r1 l1 l2)
0: -----waiting---- [7.0]
7.0: (start-moving r2 l4 l2)
7.0: -----waiting---- [11.0]
11.0: (start-moving r1 l2 l4)
11.0: (unblock-and-enter r2 l2)
11.0: (start-pick-up r2 l2 p2)
11.0: -----waiting---- [21.0]
21.0: (start-moving r2 l2 l3)
21.0: -----waiting---- [22.0]
22.0: (start-moving r1 l4 l2)
22.0: -----waiting---- [30.0]
30.0: (start-pick-up r1 l2 p1)
30.0: -----waiting---- [40.0]
40.0: (start-moving r1 l2 l3)
40.0: -----waiting---- [45.0]
45.0: (start-drop r2 p2 l3)
45.0: -----waiting---- [55.0]
55.0: (start-moving r2 l3 l2)
55.0: (unblock-and-enter r1 l3)
55.0: -----waiting---- [58.0]
58.0: (start-moving r2 l2 l3)
58.0: (start-drop r1 p1 l3)
58.0: -----waiting---- [68.0]
```

3.5.2 THREE-ROBOT BOTTLENECK

To further evaluate the domain's capacity to handle more difficult situations, a more complex scenario (`test_warehouse_three_robot`) was introduced. The spatial graph is expanded to 6 nodes. A third robot (`r3`) is added, and the packages are distributed across multiple locations (`p1` in `l6`, `p2` in `l2`, `p3` in `l4`). The packages must ultimately be delivered to different nodes (`p1` and `p2` to `l3`, and `p3` to `l5`).
This configuration creates a multi-directional bottleneck:

- **Heuristic Delay:** At $t = 0.0$, `r1` begins moving toward the central hub `l2`, arriving at $t = 10.0$. However, `r2` (located at `l4`) is intentionally held back and waits until $t = 15.0$ before moving to `l2`. The planner calculates that moving `r2` immediately would lock the intersection prematurely, stalling the heuristic descent and blocking `r1`.
- **Heuristic Plateau, Durative Blindness, and Zero-Cost Exploration:** The plan exhibits several seemingly irrational, non-productive movements, with `r2` and `r3` wandering back and forth between locations. This occurs because the GBFS planner enters a heuristic plateau caused by a combination of the domain's event-driven architecture and a lack of cost metrics. First, the pickup operation is split into an immediate start-pick-up action and a delayed end-pick-up event; since the actual heuristic reward (acquiring the package) only registers 10 ticks later, the planner evaluates start-pick-up and start-moving as having the exact same immediate value. Second, because the domain does not penalize movement with an energy or time cost, the algorithm evaluates "wandering randomly" as having the exact same cost as "waiting". Unable to see the delayed benefit of loading and facing no penalty for moving, the search tree is flooded with meaningless spatial movements until the simulated time advances enough to trigger the goal-achieving event.

- **Persistent Queueing Resolution:** The physical queue is demonstrated at the delivery node `l3` at the end of the plan. At $t = 137.0$, `r1` begins a 10-tick drop operation inside `l3`. `r2` is waiting outside with its package. Exactly at $t = 147.0$, when `r1` finishes and frees the location, the planner schedules the explicit `(unblock-and-enter r2 l3)` action, simulating a perfect waiting line.

```
Found Plan:
0: (start-moving r1 l1 l2)
0: -----waiting---- [8.0]
8.0: (start-moving r3 l3 l2)
8.0: -----waiting---- [10.0]
10.0: (start-moving r1 l2 l4)
10.0: -----waiting---- [15.0]
15.0: (start-moving r2 l4 l2)
15.0: -----waiting---- [18.0]
18.0: (start-moving r3 l2 l3)
18.0: -----waiting---- [19.0]
19.0: (start-moving r1 l4 l6)
19.0: -----waiting---- [21.0]
21.0: (start-moving r3 l3 l2)
21.0: -----waiting---- [23.0]
23.0: (start-moving r2 l2 l1)
23.0: -----waiting---- [26.0]
26.0: (start-pick-up r1 l6 p1)
26.0: -----waiting---- [36.0]
36.0: (start-moving r3 l2 l3)
36.0: (start-moving r2 l1 l2)
36.0: -----waiting---- [39.0]
39.0: (start-moving r3 l3 l2)
39.0: -----waiting---- [41.0]
41.0: (start-moving r2 l2 l3)
41.0: -----waiting---- [42.0]
42.0: (start-moving r3 l2 l3)
42.0: -----waiting---- [44.0]
44.0: (start-moving r2 l3 l5)
44.0: -----waiting---- [45.0]
45.0: (start-moving r3 l3 l5)
45.0: -----waiting---- [49.0]
49.0: (start-moving r2 l5 l3)
49.0: -----waiting---- [53.0]
53.0: (start-moving r2 l3 l2)
53.0: -----waiting---- [56.0]
56.0: (start-moving r2 l2 l4)
56.0: -----waiting---- [60.0]
60.0: (start-moving r1 l6 l4)
60.0: -----waiting---- [61.0]
61.0: (start-moving r2 l4 l2)
61.0: -----waiting---- [65.0]
```

```
65.0: (start-pick-up r2 l2 p2)
65.0: -----waiting---- [75.0]
75.0: (start-moving r2 l2 l3)
75.0: -----waiting---- [78.0]
78.0: (start-moving r1 l4 l2)
78.0: -----waiting---- [84.0]
84.0: (unblock-and-enter r3 l5)
84.0: (start-moving r3 l5 l3)
84.0: -----waiting---- [86.0]
86.0: (start-moving r2 l3 l5)
86.0: -----waiting---- [88.0]
88.0: (start-moving r3 l3 l2)
88.0: -----waiting---- [90.0]
90.0: (start-moving r1 l2 l3)
90.0: -----waiting---- [91.0]
91.0: (start-moving r3 l2 l4)
91.0: -----waiting---- [95.0]
95.0: (start-moving r3 l4 l6)
95.0: (start-moving r2 l5 l3)
95.0: -----waiting---- [97.0]
97.0: (start-moving r3 l6 l4)
97.0: -----waiting---- [99.0]
99.0: (start-pick-up r3 l4 p3)
99.0: -----waiting---- [109.0]
109.0: (start-moving r3 l4 l2)
109.0: -----waiting---- [113.0]
113.0: (start-moving r3 l2 l3)
113.0: -----waiting---- [115.0]
115.0: (start-moving r1 l3 l2)
115.0: -----waiting---- [116.0]
116.0: (start-moving r3 l3 l5)
116.0: -----waiting---- [120.0]
120.0: (start-drop r3 p3 l5)
120.0: -----waiting---- [130.0]
130.0: (start-moving r3 l5 l3)
130.0: (start-moving r1 l2 l3)
130.0: -----waiting---- [134.0]
134.0: (start-moving r3 l3 l2)
134.0: -----waiting---- [137.0]
137.0: (start-moving r3 l2 l4)
137.0: (start-drop r1 p1 l3)
137.0: -----waiting---- [147.0]
147.0: (start-moving r1 l3 l2)
147.0: (unblock-and-enter r2 l3)
147.0: (start-moving r3 l4 l2)
147.0: (start-drop r2 p2 l3)
147.0: -----waiting---- [157.0]
```

3.6 Refinement of the Blocking Mechanic: Actions vs. Events

In the early stages of the final domain, `block` was implemented as an action. However, after careful analysis of the generated plans, it was converted into an `:event`.
When `block` was an action, the planner could artificially bypass the bottleneck by hovering at the threshold (letting the movement process terminate without executing the `enter` or `block` actions, effectively "hiding" the delay). By making `block` an event, spatial constraints became physical inevitabilities. If a robot's `distance-remaining` reaches 0 and the target is `occupied`, the `block` event triggers automatically, forcing the robot into a `blocked` state.

4 Conclusion

This assignment successfully demonstrated the application of automated planning to multi-agent warehouse robotics, specifically focusing on achieving emergent coordination without explicit, hardcoded management rules.
The evaluation highlighted the fundamental differences between sequential and continuous planning paradigms. While the basic PDDL model effectively guaranteed collision-free navigation and spatial exclusivity through discrete state transitions, it abstracted away the realities of time and concurrent execution, essentially acting as a serialized traffic controller.
The transition to the continuous PDDL+ framework resolved these limitations by introducing temporal realism. By strictly defining physical boundaries through numeric fluents, continuous processes, and inevitable event triggers, the domain compelled the planner to confront real-world spatial contention.
The resulting plans for complex, multi-directional bottleneck scenarios proved the extreme efficacy of this modeling approach. Instead of relying on artificial buffers or implicit coordination loops, the heuristic search naturally generated highly sophisticated traffic-management strategies. Behaviors such as heuristic synchronization, active yielding to clear intersections, and dynamic task reallocation emerged purely as a consequence of mathematically optimizing a continuous time-space environment.